

```
In [ ]: import random

import marimo as mo
from ortools.sat.python import cp_model
from sudoku_schemas import Box, Cube, Path, SudokuConfig, SudokuSolution
```

Sudoku genérico como CSP (CP-SAT)

Esta resolução representa o Sudoku como um **problema de satisfação de restrições (CSP)**. A ideia central é separar a construção dos grupos de células da aplicação das regras: linhas, colunas, blocos e pistas são tratados pela mesma interface. As justificações seguintes explicam as escolhas observáveis na implementação, sem presumir intenções ou resultados experimentais que não estejam demonstrados nos ficheiros analisados.

Variáveis, domínios e dimensões.

Um CSP é definido por variáveis, pelos valores que podem assumir e pelas restrições que relacionam essas variáveis. Aqui, cada célula tem uma variável inteira; o seu domínio é o intervalo de 1 a n^2 ; as restrições exigem valores diferentes dentro de cada grupo e preservam os valores das células fixas.

n é o lado de cada bloco e $N = n * n$ é o lado da grelha. Não são a mesma dimensão.

n	Bloco	Grelha	Valores	Número de células
2	2×2	4×4	1 a 4	16
3	3×3	9×9	1 a 9	81
4	4×4	16×16	1 a 16	256

Existem $N^2 = n^4$ células. Usar estas dimensões em vez de constantes como 9 permite aplicar a mesma implementação a vários tamanhos. O modelo procura uma atribuição válida, sem maximizar ou minimizar qualquer expressão; por isso não tem função objetivo.

Box : representação genérica dos grupos — R1.

Em `sudoku_schemas.py`, `Box` guarda `block_size` e um dicionário `cells` que associa `(linha, coluna)` a um inteiro ou a `None`. Por exemplo, `{(0, 0): None, (0, 1): 5}` representa duas células pertencentes ao grupo: a primeira é livre e a segunda está fixa a `5`. Uma coordenada ausente do dicionário não pertence ao grupo.

Uma célula com `None` continua a participar em "todos diferentes". Só não tem uma igualdade que a fixe a um valor. Esta distinção evita confundir pertença ao grupo com existência de uma pista.

O dicionário permite representar grupos pequenos ou irregulares sem guardar uma matriz inteira. As suas chaves coincidem com as chaves de `SudokuSolver.x`, permitindo obter diretamente a variável de cada célula. `Box` guarda os dados; a restrição matemática é acrescentada posteriormente pelo solver.

A classe herda de `BaseModel`, do Pydantic. `Field(ge=1)` exige um tamanho de bloco positivo, e `@model_validator(mode="after")` verifica as células fornecidas ao construtor. A propriedade `N` calcula `block_size**2`, evitando guardar duas dimensões que poderiam ficar inconsistentes.

`_validar` exige coordenadas entre `0` e `N - 1` e valores fixos entre `1` e `N`. `add(i, j, val=None)` chama essa validação antes de atualizar o dicionário. A expressão `self.cells[i, j]` usa a tupla `(i, j)` como chave. Adicionar novamente a mesma posição substitui o valor anterior: não guarda duas atribuições em conflito.

Embora `cells` esteja declarado com `{}` por omissão, os objetos Pydantic criados separadamente têm dicionários independentes neste caso. Não se deve confundir este comportamento com o problema dos argumentos mutáveis por omissão em funções Python. `Field(default_factory=dict)` seria uma alternativa mais explícita.

`representation()` cria uma matriz com zeros e coloca os valores fixos nas posições correspondentes. `[[0] * N for _ in range(N)]` cria linhas independentes; usar `[[0] * N] * N` poderia fazer todas as linhas partilharem a mesma lista. `val or 0` funciona porque os inteiros válidos são positivos. A matriz é apenas uma representação: tanto células livres do grupo como posições ausentes aparecem com zero.

Cube : construção dos blocos — R2.

`Cube` herda de `Box` e recebe índices de bloco `(i, j)`, com `0 <= i, j < n`. O canto superior esquerdo é a célula `(i * n, j * n)`. Por exemplo, `Cube(1, 2, 3)` contém as linhas `3, 4, 5` e as colunas `6, 7, 8`.

Os ciclos usam `range(i * n, (i + 1) * n)` e o equivalente para as colunas. Como o limite superior de `range` é exclusivo, cada ciclo percorre exatamente `n` posições e o bloco contém `n2` células. As células são acrescentadas por `self.add`, reutilizando a validação de `Box`, inicialmente sem valores fixos.

Path : construção dos segmentos — R3.

`Path` representa um segmento horizontal ou vertical, incluindo os extremos. Se a linha e a coluna mudarem simultaneamente, a condição `i0 != i1 and j0 != j1` rejeita o segmento. Um percurso horizontal mantém a linha; um vertical mantém a coluna.

As expressões `(i1 > i0) - (i1 < i0)` e `(j1 > j0) - (j1 < j0)` calculam os passos `di` e `dj`. Os booleanos valem `0` ou `1` nestas operações, pelo que cada passo é `-1`, `0` ou `1`. Isto permite percorrer o segmento nos dois sentidos.

De `(2, 4)` para `(2, 1)`, por exemplo, `di = 0` e `dj = -1`, produzindo `(2, 4)`, `(2, 3)`, `(2, 2)`, `(2, 1)`. O número de células é `max(abs(i1 - i0), abs(j1 - j0)) + 1`: o `+1` inclui os dois extremos. Se início e fim coincidirem, o grupo contém uma única célula.

Estas subclasses sabem construir coordenadas específicas. O armazenamento e a validação mantêm-se em `Box`, e a aplicação das restrições mantêm-se no solver.

grupos_sudoku : composição da grelha — R6.

Cada linha é um `Path((k, 0), (k, N - 1), n)` e cada coluna é um `Path((0, k), (N - 1, k), n)`. Os blocos são os `Cube(i, j, n)` para todos os pares de índices de bloco. São criados `N` grupos de linhas, `N` de colunas e `n2` de blocos: no total, `3n2` grupos. Para `n = 3`, são 27.

`[*linhas, *colunas, *blocos]` expande as três listas numa única lista de grupos. A função define que células ficam relacionadas; o solver define a regra dessas relações.

pistas_aleatorias : posições e valores sem repetição — R4.

`random.Random(seed)` cria um gerador local. Uma semente fixa permite reproduzir as pistas e evita depender do estado do gerador aleatório global. `k = n if k is None else k` escolhe `n` pistas por omissão e preserva `k = 0`.

`rng.sample` escolhe `k` posições sem reposição entre as `N2` células. Uma segunda chamada escolhe `k` valores sem reposição entre `1` e `N`. `dict(zip(celulas, valores, strict=True))` associa cada posição a um valor; `strict=True` rejeitaria sequências de comprimentos diferentes.

Os valores distintos são necessários porque, segundo a interface do enunciado, o **grupo das pistas também recebe "todos diferentes"**. Duas pistas com o mesmo valor no mesmo `Box` tornariam o modelo impossível, mesmo que essas posições fossem compatíveis num Sudoku habitual. Por isso, o intervalo permitido é `0 <= k <= N`: existem `N2` posições, mas apenas `N` valores distintos para este grupo. O resultado é outro `Box`, sem uma classe especial para pistas.

SudokuSolver : variáveis e motor de resolução — R5.

`CpModel` guarda as variáveis e as restrições; `CpSolver` executa a pesquisa. CP-SAT é adequado a esta formulação porque trabalha com variáveis inteiras e permite expressar diretamente "todos diferentes" através de `add_all_different`. A implementação específica o problema e deixa a pesquisa a cargo da biblioteca.

O dicionário `self.x` cria uma variável por posição através de `new_int_var(1, self.N, f"x_{i}_{j}")`. Para `n = 3`, são 81 variáveis com domínio `1..9`. A chave é a coordenada e o nome, como `x_0_0`, ajuda a identificar a variável. Criar estas variáveis, por si só, ainda não impõe linhas, colunas ou blocos.

`SudokuConfig` reúne o limite de tempo e a opção de registar o progresso.

`SudokuSolution` reúne o estado, o tempo reportado pelo solver e uma matriz opcional. A separação destes dados da resolução facilita a utilização e os testes.

add : tradução uniforme dos grupos em restrições.

`add(self, *groups)` recebe um número arbitrário de grupos. Primeiro verifica se `g.N == self.N`, rejeitando grupos de outra dimensão. Depois acrescenta `add_all_different(self.x[c] for c in g.cells)` e, para cada valor não nulo, a igualdade `self.x[c] == val`.

Uma célula presente numa linha, numa coluna e num bloco corresponde sempre à mesma variável. É essa partilha que faz as restrições interagirem. Para um grupo de células G , "todos diferentes" significa $x_a \neq x_b$ para quaisquer células distintas $a, b \in G$. Uma pista com valor v acrescenta $x_a = v$.

Uma linha contém `N` variáveis diferentes, cada uma no domínio `1..N`. Como existem exatamente `N` valores disponíveis, a linha tem necessariamente cada valor uma vez. O mesmo raciocínio vale para colunas e blocos. Não é necessária uma restrição adicional para exigir a presença de todos os números.

O método não distingue `Box`, `Cube` e `Path`: usa apenas a dimensão e as células. Assim, uma variante com diagonais pode acrescentar novos `Box` sem alterar o modelo. As restrições acrescentadas ficam acumuladas; alterar um grupo depois de o adicionar não atualiza automaticamente as restrições já construídas.

solve : pesquisa, estados e extração da matriz.

O método configura os parâmetros, executa `self.solver.solve(self.model)` e guarda o nome do estado e `wall_time`. Este tempo é o reportado pelo solver, não o tempo total de geração das pistas, construção dos grupos e execução do notebook.

Estado	Interpretação
OPTIMAL	Solução com conclusão de optimalidade; sem objetivo, não é uma grelha melhor.
FEASIBLE	Foi encontrada uma solução válida.
INFEASIBLE	Foi provado que o modelo não admite solução.
UNKNOWN	Não foi encontrada solução nem provada a impossibilidade antes da paragem.
MODEL_INVALID	O modelo não passou a validação do solver.

`UNKNOWN` pode resultar de limites de tempo, memória ou outras condições de paragem; não significa obrigatoriamente tempo esgotado. Esta distinção é descrita na [documentação oficial do CP-SAT](#).

Só nos estados `OPTIMAL` ou `FEASIBLE` são consultados os valores das variáveis. `self.solver.value(self.x[i, j])` extrai o inteiro atribuído a cada célula e os dois ciclos constroem a matriz linha a linha. Nos restantes estados, `matrix` mantém-se `None`. É necessário consultar `status` para distinguir as razões dessa ausência.

sudoku_aleatorio : coordenação e novas tentativas.

Cada tentativa gera pistas, cria um novo `SudokuSolver`, acrescenta os grupos e resolve. Em `s.add(*grupos_sudoku(n), pistas)`, o `*` passa cada grupo da lista como argumento individual e as pistas entram como mais um grupo.

Criar um solver novo evita acumular pistas de tentativas anteriores. O gerador externo produz sementes com `rng.randrange(2**32)`, permitindo reproduzir a sequência de pistas a partir de uma semente inicial. Isso não garante tempos ou grelhas devolvidas pelo solver sempre idênticos.

O ciclo termina na primeira matriz encontrada. Caso nenhuma tentativa produza uma, devolve as últimas pistas e o último resultado. A repetição ocorre sempre que não há matriz, incluindo `UNKNOWN`, e não apenas quando há prova de impossibilidade. O limite de tempo aplica-se a cada resolução: dez tentativas de 30 segundos podem aproximar-se de 300 segundos, além do tempo de preparação.

mostrar e apresentação no Marimo.

`mostrar` transforma a matriz em texto. `len(str(len(sol.matrix)))` calcula a largura necessária para os números: uma posição para uma grelha 9×9 , duas para 16×16 . `f"{v:>{largura}}"` alinha os valores à direita. Um `join` separa os valores com espaços e outro separa as linhas com mudanças de linha.

A célula final apresenta exemplos para `n = 3` e `n = 2`, com pistas, estado, tempo e grelha. A verificação automática das regras está nos testes, não nessa apresentação.

`app.setup` reúne os imports; `@app.function` e `@app.class_definition` registam definições no notebook; `@app.cell` identifica células de documentação e apresentação. `if __name__ == "__main__": app.run()` inicia a aplicação na execução direta, sem executar essa chamada quando o módulo é importado pelos testes.

Validação realizada e limites da implementação.

Na análise desta versão, os **42 testes de Sudoku passaram isoladamente**. A execução normal ficou bloqueada por uma importação de `HorarioSolver` no `tests/conftest.py`, relacionada com o trabalho dos horários. A execução isolada utilizou uma cópia dos testes e a constante `VIAVEL`, sem alterar os testes do projeto.

O verificador `_valida` calcula diretamente linhas, colunas e blocos da matriz, sem reutilizar `Cube` ou `Path`. Esta independência reduz o risco de repetir no verificador um erro da construção dos grupos. Os testes cobrem também valores e coordenadas inválidos, percursos inversos, domínios, conflitos, preservação de pistas e resolução completa para `n = 2` e `n = 3` com várias sementes.

Foram identificados os seguintes pontos que os testes principais não resolvem:

- **Número de tentativas:** `tentativas <= 0` não executa o ciclo; `pistas` e `sol` ficam por definir e o retorno levanta `UnboundLocalError`. Deve exigir-se pelo menos uma tentativa.
- **Limite zero:** `time_limit or self.config.time_limit` substitui `0` pelo limite da configuração. Para distinguir ausência de argumento de zero, deve verificar-se explicitamente `time_limit is None` e validar os limites aceites.
- **Mensagem de insucesso:** `mostrar` escreve "Sem solução" também para `UNKNOWN`. Nesse estado, "Não foi obtida uma solução" seria mais preciso, pois não houve prova de impossibilidade.
- **Validação dos parâmetros:** `SudokuSolver` não valida diretamente `n`; `SudokuSolver(-3)` calcula `N = 9`. `Box` exige tamanho positivo, mas a validação não é uniforme. Valores inválidos de `k` são rejeitados por `random.sample`, embora uma verificação explícita pudesse produzir uma mensagem mais clara. Anotações de tipos, por si só, não validam argumentos de funções Python comuns.
- **Mutação dos dados:** a validação na construção e em `Box.add` não impede que alguém altere diretamente `cells` com dados inválidos. A utilização normal deve respeitar a interface de validação.
- **Unicidade e dificuldade:** o código encontra uma solução, mas não pesquisa uma segunda para verificar unicidade nem classifica a dificuldade para um jogador.
- **Hints e desempenho:** esta versão não usa `add_hint`. A documentação anterior afirmava não haver ganho mensurável com hints para `n = 3` a `5`, mas os

ficheiros de Sudoku e testes analisados não contêm o ensaio nem os resultados dessa comparação. Essa conclusão experimental precisa de evidência própria.

A correção central assenta numa propriedade simples: **cada célula tem uma única variável; cada grupo seleciona algumas dessas variáveis; o modelo impõe valores diferentes e preserva as pistas.** A combinação de todas as linhas, colunas e blocos transforma esse mecanismo genérico num Sudoku completo.

```
In [ ]: def grupos_sudoku(n: int) -> list[Box]:
        """R6 - todas as linhas, colunas e blocos n x n de uma grelha n² x
        n²."""
        N = n * n
        linhas = [Path((k, 0), (k, N - 1), n) for k in range(N)]
        colunas = [Path((0, k), (N - 1, k), n) for k in range(N)]
        blocos = [Cube(i, j, n) for i in range(n) for j in range(n)]
        return [*linhas, *colunas, *blocos]
```

```
In [ ]: def pistas_aleatorias(n: int = 3, k: int | None = None, seed: int | None =
None) -> Box:
    """R4 - k células aleatórias (por omissão k = n), fixas a valores
aleatórios distintos."""
    rng = random.Random(seed)
    N = n * n
    k = n if k is None else k
    celulas = rng.sample([i, j] for i in range(N) for j in range(N)), k)
    valores = rng.sample(range(1, N + 1), k)
    return Box(block_size=n, cells=dict(zip(celulas, valores,
strict=True)))
```

```
In [ ]: class SudokuSolver:
    def __init__(self, n: int = 3, config: SudokuConfig | None = None):
        self.N = n * n
        self.config = config or SudokuConfig()
        self.model = cp_model.CpModel()
        # R5 - uma variável inteira por célula, em [1, n²]
        self.x = {
            (i, j): self.model.new_int_var(1, self.N, f"x_{i}_{j}")
            for i in range(self.N)
            for j in range(self.N)
        }
        self.solver = cp_model.CpSolver()

    def add(self, *groups: Box) -> None:
        # R5 - para cada grupo, seja qual for a origem: todos diferentes +
        # células fixas
        for g in groups:
            if g.N != self.N:
                raise ValueError(f"Grupo para grelha {g.N}x{g.N}, modelo é
{self.N}x{self.N}")
            self.model.add_all_different(self.x[c] for c in g.cells)
            for c, val in g.cells.items():
                if val is not None:
                    self.model.add(self.x[c] == val)
```

```

    def solve(self, time_limit: float | None = None) -> SudokuSolution:
        self.solver.parameters.max_time_in_seconds = time_limit or
self.config.time_limit
        self.solver.parameters.log_search_progress =
self.config.log_search_progress
        status = self.solver.solve(self.model)
        sol = SudokuSolution(status=self.solver.status_name(status),
tempo=self.solver.wall_time)
        if status in (cp_model.OPTIMAL, cp_model.FEASIBLE):
            sol.matrix = [
                [self.solver.value(self.x[i, j]) for j in range(self.N)]
for i in range(self.N)
                ]
            return sol

```

```

In [ ]: def sudoku_aleatorio(
        n: int = 3, k: int | None = None, seed: int | None = None, tentativas:
        int = 10
    ) -> tuple[Box, SudokuSolution]:
        """Gera pistas → monta linhas + colunas + blocos + pistas → resolve.

        Se as pistas não tiverem solução, tenta novas (até `tentativas`) e
        devolve a última."""
        rng = random.Random(seed)
        for _ in range(tentativas):
            pistas = pistas_aleatorias(n, k, rng.randrange(2**32))
            s = SudokuSolver(n)
            s.add(*grupos_sudoku(n), pistas)
            sol = s.solve()
            if sol.matrix is not None:
                break
        return pistas, sol

```

```

In [ ]: def mostrar(sol: SudokuSolution) -> str:
        if sol.matrix is None:
            return f"Sem solução ({sol.status})"
        largura = len(str(len(sol.matrix)))
        return "\n".join(" ".join(f"{v:>{largura}}" for v in linha) for linha
in sol.matrix)

```

```

In [ ]: pistas_3, sol_3 = sudoku_aleatorio(n=3, seed=0)
        pistas_2, sol_2 = sudoku_aleatorio(n=2, seed=0)
        mo.vstack(
            [
                mo.md(f"*n = 3* ({sol_3.status}, {sol_3.tempo:.3f}s), pistas
`{pistas_3.cells}`"),
                mo.plain_text(mostrar(sol_3)),
                mo.md(f"*n = 2* ({sol_2.status}, {sol_2.tempo:.3f}s), pistas
`{pistas_2.cells}`"),
                mo.plain_text(mostrar(sol_2)),
            ]
        )

```

n = 3 (OPTIMAL, 0.048s), pistas $\{(8, 0): 6, (1, 7): 3, (7, 8): 9\}$

8 6 2 4 1 3 7 9 5

9 4 1 5 8 7 6 3 2

3 5 7 2 6 9 1 8 4

2 8 5 1 4 6 9 7 3

4 3 6 9 7 5 2 1 8

7 1 9 3 2 8 4 5 6

5 2 3 6 9 1 8 4 7

1 7 4 8 3 2 5 6 9

6 9 8 7 5 4 3 2 1

n = 2 (OPTIMAL, 0.005s), pistas $\{(1, 0): 3, (2, 2): 4\}$

1 4 3 2

3 2 1 4

2 3 4 1

4 1 2 3